

POSTGRESQL

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU QUAN HỆ MÃ NGUỒN MỞ

Ninh Văn Khải — 23127060

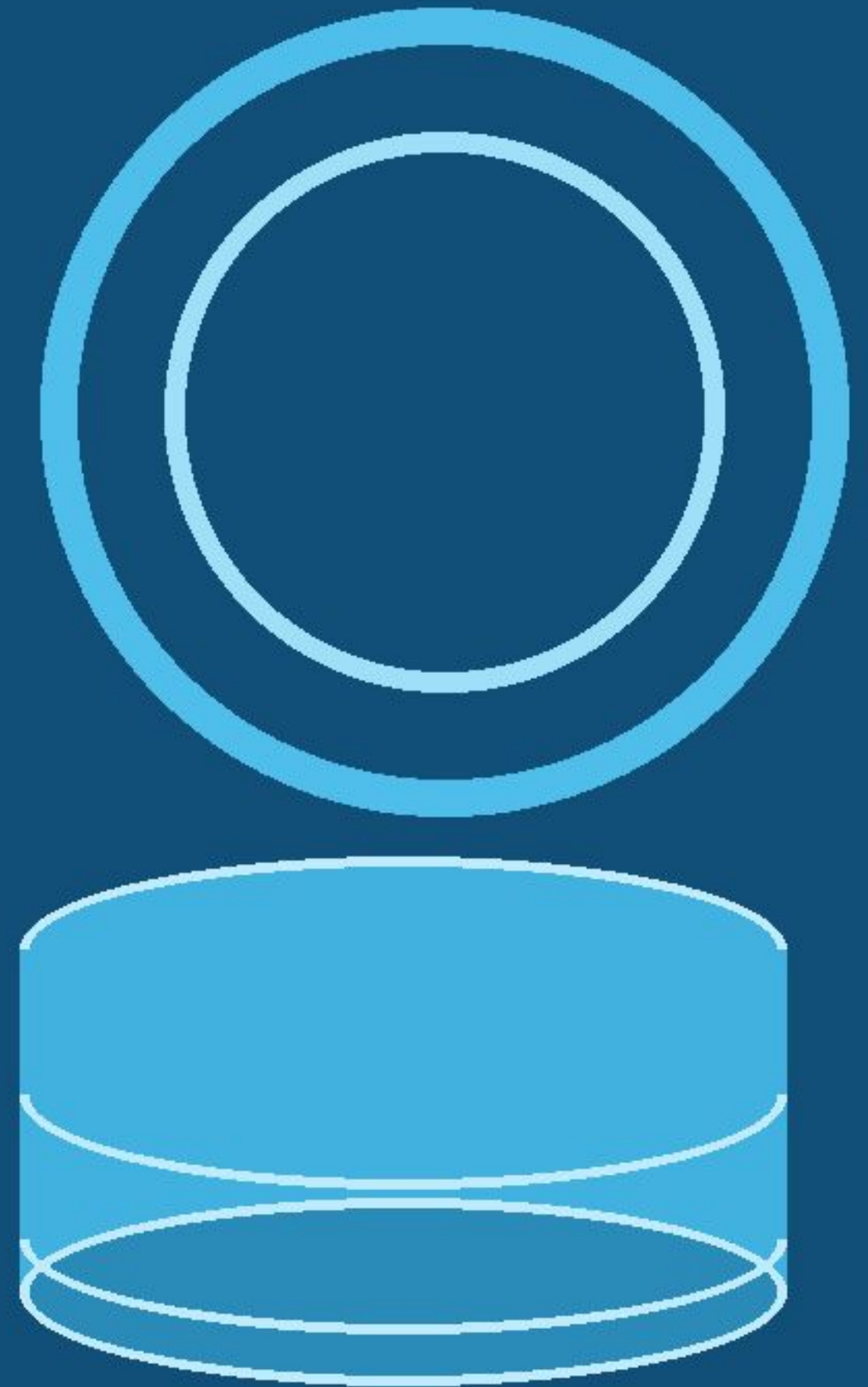
Nguyễn Tấn Thắng — 23127259

Kiều Duy Hiếu — 23127365

Đình Tuấn Duy — 23127356

Nguyễn Tấn Phước — 23127536

Đỗ Hiếu Quang — 23127537



Agenda

AGENDA

01 NỀN TẢNG

- Vấn đề
- RDBMS
- PostgreSQL

02 KHÁI NIỆM

- Thuật ngữ
- Kiến trúc
- Kết nối

03 CÀI ĐẶT & CẤU HÌNH

- Ubuntu
- postgresql.conf
- pg_hba.conf

04 DEMO CƠ BẢN

- Role
- Database
- Table
- CRUD

05 HIỆU NĂNG & VẬN HÀNH

- Index
- Backup/Restore
- Xử lý lỗi

06 LỰA CHỌN & KẾT LUẬN

- So sánh
- Khi nên dùng
- Takeaways

NỀN TẢNG → TRIỂN KHAI → DEMO → VẬN HÀNH → LỰA CHỌN

Vì sao cần RDBMS?

NỀN TẢNG

01 MÔ HÌNH HÓA DỮ LIỆU

Bảng, hàng và cột giúp biểu diễn quan hệ 1-1, 1-n rõ ràng.

02 TOÀN VỆN DỮ LIỆU

PRIMARY KEY, FOREIGN KEY và NOT NULL ngăn dữ liệu sai hoặc mâu thuẫn.

03 GIAO DỊCH ACID

Atomicity: thành công toàn bộ hoặc hủy toàn bộ. Durability: dữ liệu đã lưu không bị mất.

04 TỐI ƯU TRUY VẤN

SQL lọc, trích xuất và thống kê dữ liệu lớn thay vì tự duyệt từng dòng.

Vì sao chọn PostgreSQL?

NỀN TẢNG

01 JSON / JSONB

Linh hoạt như Document Store nhưng vẫn nằm trong hệ thống quan hệ.

02 MVCC

Đọc và ghi có thể diễn ra song song, giảm nghẽn khi tải lớn.

03 ANSI SQL

Bám sát chuẩn quốc tế, dễ bảo trì và giảm vendor lock-in.

04 MÃ NGUỒN MỞ & MỞ RỘNG

Giàu kiểu dữ liệu, extension và phù hợp từ ứng dụng nhỏ đến hệ thống phức tạp.

Thuật ngữ cốt lõi

NỀN TẢNG

RDBMS — phần mềm quản lý cơ sở dữ liệu theo mô hình quan hệ

DATABASE — không gian độc lập chứa schema, table và index

PSQL — CLI để kết nối, chạy SQL, debug và quản trị

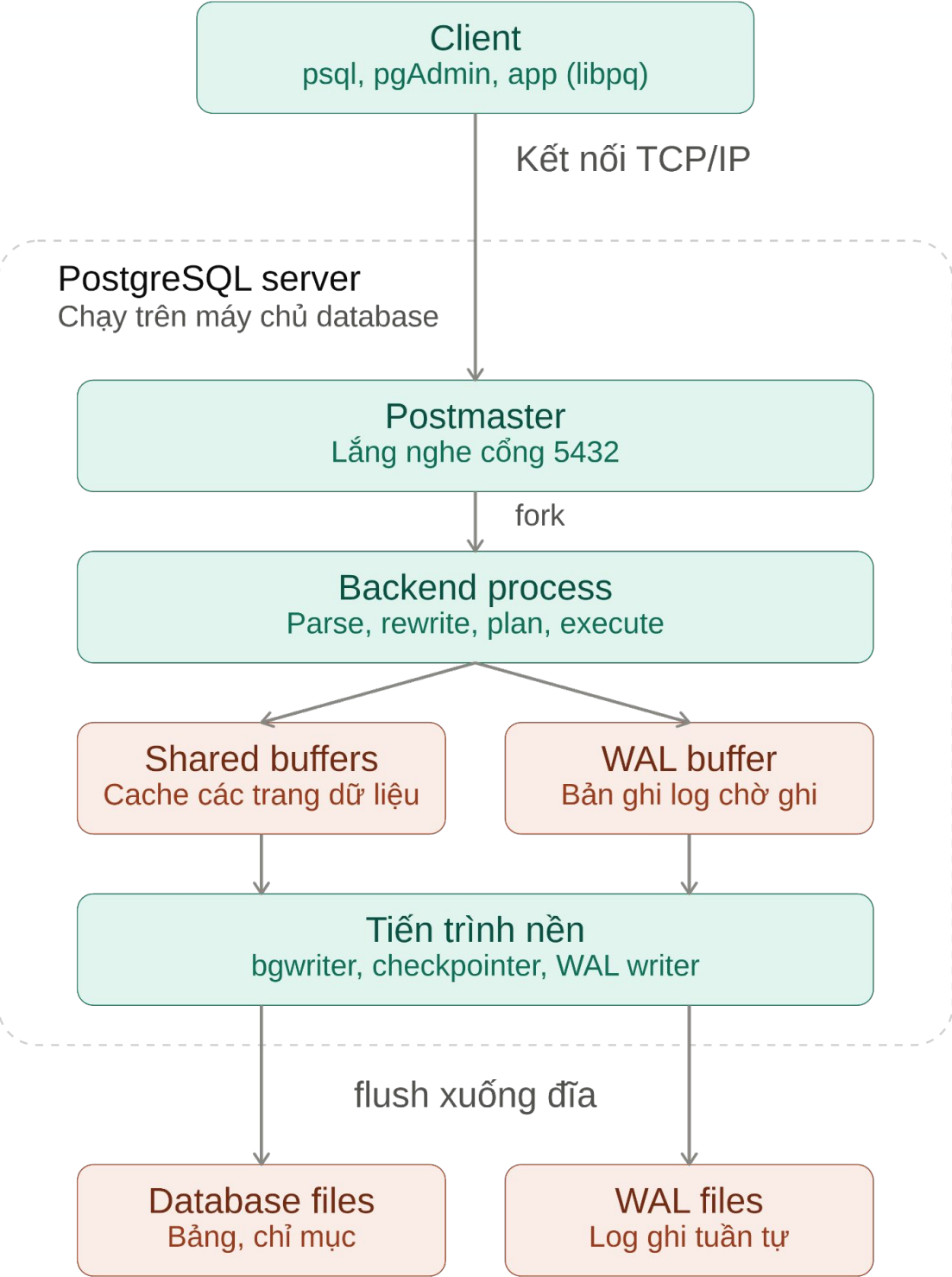
BACKUP / RESTORE — tạo bản dự phòng bằng pg_dump và khôi phục khi xảy ra sự cố

ROLE / USER — role có LOGIN hoạt động như người dùng

TABLE — cột định nghĩa cấu trúc; hàng lưu từng bản ghi

INDEX — “mục lục” giúp truy vấn tránh quét toàn bảng

Kiến trúc tổng thể PostgreSQL



Thành phần chính của kiến trúc

Client → Gửi yêu cầu truy vấn SQL

Postmaster → Lắng nghe và tiếp nhận kết nối TCP/IP

Backend process → Xử lý, tối ưu hóa và thực thi SQL

Shared buffers → Vùng đệm để bộ nhớ cache dữ liệu

WAL (Write-Ahead Log) → Ghi nhật ký trước để khôi phục dữ liệu

Ba lớp kiểm soát kết nối

KIẾN TRÚC & CẤU HÌNH

01

NETWORK Postgresql.conf

- Nơi lắng nghe
- Cổng & RAM
 - Log
 - WAL

02

CONNECTION RULES pg_hba.conf

- Database
 - Role
 - IP
- Phương thức xác thực

03

AUTHENTICATION & AUTHORIZATION

- username/password
 - role/grant

LẮNG NGHE → XÁC THỰC → PHÂN QUYỀN → THỰC THI SQL

Cài đặt trên Ubuntu / Debian

KIẾN TRÚC & CẤU HÌNH



CÀI ĐẶT

Ubuntu / Debian — Terminal

```
$ sudo apt update
$ sudo apt install postgresql postgresql-contrib

$ sudo systemctl status postgresql
→ active (running)

$ sudo systemctl enable postgresql
$ sudo systemctl start postgresql
$ sudo systemctl stop postgresql
$ sudo systemctl restart postgresql
```

✓ PostgreSQL đã cài, service đang chạy và tự khởi động cùng hệ thống.

postgresql.conf - File cấu hình chính

KIẾN TRÚC & CẤU HÌNH

```
# - Connection Settings -

#listen_addresses = 'localhost'          # what IP address(es) to listen on;
                                           # comma-separated list of addresses;
                                           # defaults to 'localhost'; use '*' for all
                                           # (change requires restart)
port = 5432                               # (change requires restart)
max_connections = 100                     # (change requires restart)

# - Memory -

shared_buffers = 128MB                    # min 128kB
                                           # (change requires restart)
```

listen_addresses = nghe ở IP nào | port = cổng kết nối | max_connections = số kết nối đồng thời | shared_buffers = RAM đệm dữ liệu

pg_hba.conf - File định nghĩa các rule xác thực kết nối

KIẾN TRÚC & CẤU HÌNH



```
# - Client Authentication Rules -  
  
# TYPE      DATABASE  USER      ADDRESS      METHOD  
local       all       all          
host        all       all        127.0.0.1/32  scram-sha-256  
host        all       all        192.168.1.0/24 scram-sha-256  
  
# local: Unix socket (cùng máy, không có ADDRESS)  
# host:  TCP/IP (có ADDRESS/IP)  
# all:   mọi database / mọi role  
# peer: tài khoản Linux | scram-sha-256: xác thực bằng mật khẩu
```

TYPE = loại kết nối | DATABASE = DB áp dụng | USER = role áp dụng | ADDRESS = IP client | METHOD = cách xác thực

Luồng kết nối end-to-end

KIẾN TRÚC & CẤU HÌNH



Client được tiếp nhận → xác thực danh tính → kiểm tra quyền → PostgreSQL thực thi và trả kết quả.
Mỗi lớp có một trách nhiệm riêng, giúp kết nối an toàn và dễ kiểm soát.

Demo – kiểm tra môi trường

DEMO CRUD

```
ubuntu@postgres-demo:~$ sudo systemctl status postgresql --no-pager
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
   Active: active (exited) since Fri 2026-07-31 13:37:11 UTC; 1min 0s ago
     Main PID: 4703 (code=exited, status=0/SUCCESS)
        CPU: 1ms
```

01 SERVER

postgresql.service đã được bật

02 TRẠNG THÁI

Service trả mã SUCCESS

03 XÁC NHẬN

Kết nối thật bằng psql ở bước tiếp theo

TIẾP THEO: DÙNG psql ĐỂ XÁC NHẬN KẾT NỐI THẬT TỚI DATABASE QUA CỔNG 5432

Demo — tạo Role & Database

DEMO CRUD

```
postgres=# SELECT
  rolname AS role_name,
  rolcanlogin AS can_login
FROM pg_roles
WHERE rolname = 'demo_user';
 role_name | can_login 
-----+-----
demo_user | t
(1 row)

postgres=# SELECT
  datname AS database_name,
  pg_get_userbyid(datdba) AS owner
FROM pg_database
WHERE datname = 'demo_crud';
 database_name | owner 
-----+-----
demo_crud      | demo_user
(1 row)
```

ROLE

demo_user
can_login = true

DATABASE

demo_crud
owner = demo_user

Role có LOGIN và sở hữu
database demo.

Demo – đăng nhập bằng Role mới

DEMO CRUD

```
demo_crud=> \conninfo  
You are connected to database "demo_crud" as user "demo_user" on host "127.0.0.1" at port "5432".  
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
```

demo_user → demo_crud → 127.0.0.1:5432 → SSL / TLS

Role đăng nhập thành công và PostgreSQL xác nhận đúng database, host và cổng kết nối.

DẤU NHẮC demo_crud=> CHO BIẾT ĐANG LÀM VIỆC TRONG DATABASE BẰNG ROLE MỚI

Demo – tạo Table

DEMO CRUD

```
demo_crud=> \dt
              List of relations
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 public | departments    | table | demo_user
 public | employees       | table | demo_user
(2 rows)
```

departments
Lưu danh sách phòng ban

employees
Lưu nhân viên và khóa ngoại tới departments

CẢ HAI BẢNG ĐỀU THUỘC OWNER demo_user

Mỗi ràng buộc ngăn lỗi gì?

DEMO CRUD

1 BẢNG PHẢI HỢP LỆ

- PRIMARY KEY: mỗi hàng có một mã duy nhất
- NOT NULL: trường bắt buộc không được để trống

2 GIÁ TRỊ PHẢI ĐÚNG

- UNIQUE: email và tên phòng ban không được trùng
- CHECK: lương phải lớn hơn 0

3 QUAN HỆ PHẢI TỒN TẠI

- FOREIGN KEY: department_id phải trỏ tới phòng ban có thật
- RESTRICT: không xóa phòng ban đang có nhân viên

VÍ DỤ: QUẢN LÝ NHÂN VIÊN

DEPARTMENTS

id = 1
name = 'IT'

1 → N

EMPLOYEES

id = 101
name = 'An'
department_id = 1

employees.department_id phải trùng với một departments.id có thật.

BỊ TỪ CHỐI: thêm nhân viên có department_id = 99 khi không tồn tại phòng ban 99.

DATABASE TỰ CHẶN DỮ LIỆU SAI – ỨNG DỤNG KHÔNG PHẢI TỰ KIỂM TRA MỌI TRƯỜNG HỢP

CRUD – INSERT

DEMO CRUD



CRUD

INSERT — thêm dữ liệu

```
INSERT INTO departments (name)
VALUES ('Engineering'), ('Human Resources')
RETURNING *;

INSERT INTO employees (full_name, email, salary, department_id)
VALUES
  ('Nguyen Van An', 'an.nguyen@example.com', 15000000, 1),
  ('Tran Thi Binh', 'binh.tran@example.com', 12000000, 2)
RETURNING *;
```

✓ INSERT 0 2 — tạo thành công 2 phòng ban và 2 nhân viên.

CRUD – SELECT với JOIN

DEMO CRUD

id	full_name	email	department	salary
1	Nguyen Van An	an.nguyen@example.com	Engineering	15000000.00
2	Tran Thi Binh	binh.tran@example.com	Human Resources	12000000.00
3	Le Minh Chau	chau.le@example.com	Engineering	18000000.00
4	Pham Quang Duy	duy.pham@example.com	Sales	14000000.00
5	CRUD Demo User	crud.demo@example.com	Engineering	11000000.00

(5 rows)

EMPLOYEES
id, full_name, email, salary

JOIN
employees.department_id =
departments.id

KẾT QUẢ
Tên phòng ban xuất hiện cùng nhân
viên

SELECT + JOIN BIẾN KHÓA NGOẠI department_id THÀNH TÊN PHÒNG BAN DỄ ĐỌC

CRUD – UPDATE & DELETE

DEMO CRUD



CRUD

UPDATE & DELETE

UPDATE employees

```
SET salary = salary * 1.10
```

```
WHERE email = 'an.nguyen@example.com'
```

```
RETURNING id, full_name, salary;
```

```
→ UPDATE 1 | 15,000,000 → 16,500,000
```

DELETE FROM employees

```
WHERE email = 'binh.tran@example.com'
```

```
RETURNING id, full_name, email;
```

✓ Sau DELETE, bảng employees chỉ còn Nguyen Van An.

Kiểm thử tính toàn vẹn

DEMO CRUD

```
demo_crud=> INSERT INTO employees (  
    full_name,  
    email,  
    salary,  
    department_id  
)  
VALUES (  
    'Invalid Salary User',  
    'invalid.salary@example.com',  
    -1000,  
    (SELECT id FROM departments WHERE name = 'Engineering')  
);  
ERROR:  new row for relation "employees" violates check constraint "employees_salary_check"  
DETAIL:  Failing row contains (6, Invalid Salary User, invalid.salary@example.com, -1000.00, 1).
```

POSTGRESQL TỪ CHỐI BẢN GHI: salary = -1000 VI PHẠM CHECK (salary > 0)

RÀNG BUỘC BẢO VỆ DỮ LIỆU NGAY TRONG DATABASE – KHÔNG PHỤ THUỘC HOÀN TOÀN VÀO ỨNG DỤNG

Tạo Index đúng cột: department_id

HIỆU NĂNG & VẬN HÀNH



01 KHÔNG DÙNG email

- email đã khai báo UNIQUE
- PostgreSQL tự tạo index cho UNIQUE
- Tạo thêm index email sẽ bị trùng chức năng

02 CHỌN department_id

- Dùng để lọc nhân viên theo phòng ban
- Dùng nhiều trong JOIN
- Phù hợp hơn để demo CREATE INDEX

03 KIỂM TRA BẰNG EXPLAIN

- Chạy truy vấn trước / sau index
- Xem Seq Scan hay Index Scan
- Bảng nhỏ vẫn có thể chọn Seq Scan

KẾT QUẢ THẬT: INDEX ĐÃ ĐƯỢC TẠO TRÊN BẢNG employees

```
demo_crud=> \di idx_employees_department_id
              List of relations
 Schema |          Name          | Type  | Owner  | Table
-----+-----+-----+-----+-----
 public | idx_employees_department_id | index | demo_user | employees
(1 row)
```


Backup bằng pg_dump

HIỆU NĂNG & VẬN HÀNH

```
ubuntu@postgres-demo:~/postgres-demo-backup$ pg_dump -h 127.0.0.1 -U demo_user -d demo_crud -Fc -f postgres_demo.dump -W
Password:
ubuntu@postgres-demo:~/postgres-demo-backup$ ls -lh postgres_demo.dump
-rw-rw-r-- 1 ubuntu ubuntu 5.3K Jul 31 13:52 postgres_demo.dump
```

01 TẠO BACKUP

pg_dump -Fc
Custom format

02 KIỂM TRA FILE

postgres_demo.dump
5.3 KB > 0

03 SẴN SÀNG RESTORE

Dùng pg_restore
vào database mới

FILE BACKUP ĐÃ ĐƯỢC TẠO THÀNH CÔNG VÀ CÓ DỮ LIỆU

Restore thành công vào database mới

HIỆU NĂNG & VẬN HÀNH

RESTORE THÀNH CÔNG

- 01 Kết nối database demo_crud_restore
- 02 Hai bảng departments và employees xuất hiện
- 03 COUNT(*) trả về 4 nhân viên
- 04 JOIN trả lại đúng phòng ban và mức lương

```
demo_crud_restore=> \set pager off
\conninfo
\dt
Pager usage is off.
You are connected to database "demo_crud_restore" as user "demo_user" on host "127.0.0.1" at port "5432".
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | departments | table | demo_user
public | employees | table | demo_user
(2 rows)

demo_crud_restore=> SELECT COUNT(*) AS restored_employees
FROM employees;
restored_employees
-----
4
(1 row)

demo_crud_restore=> SELECT
e.full_name,
d.name AS department,
e.salary
FROM employees e
JOIN departments d
ON e.department_id = d.id
ORDER BY e.id
LIMIT 5;
full_name | department | salary
-----+-----+-----
Nguyen Van An | Engineering | 15000000.00
Tran Thi Binh | Human Resources | 12000000.00
Le Minh Chau | Engineering | 18000000.00
Pham Quang Duy | Sales | 14000000.00
(4 rows)
```

BACKUP → DATABASE MỚI → pg_restore → KIỂM TRA BẢNG VÀ DỮ LIỆU

Xử lý kết nối bị từ chối

HIỆU NĂNG & VẬN HÀNH



BONUS

Sửa lỗi kết nối bị từ chối

FATAL: no pg_hba.conf entry for host ...

Thêm rule đúng host / user / method

host all app_user 192.168.1.0/24 scram-sha-256

\$ sudo systemctl reload postgresql

hoặc

postgres=# SELECT pg_reload_conf();

✓ Reload cấu hình, sau đó client kết nối lại thành công.

PostgreSQL vs MySQL vs MariaDB

SO SÁNH & BEST PRACTICES

POSTGRESQL

OBJECT-RELATIONAL

- Sát chuẩn ANSI SQL
- JSONB, mảng, range, ENUM
- Kiểu tự định nghĩa và PostGIS
- MVCC mạnh: đọc/ghi ít khóa nhau

MYSQL

RELATIONAL, GỌN NHẸ

- Có một số sai lệch / nói lỏng SQL
- Kiểu dữ liệu cơ bản và JSON
- JSON kém linh hoạt hơn JSONB
- MVCC thông qua InnoDB

MARIADB

FORK CỦA MYSQL

- Mức tương thích với MySQL cao
- Chuẩn SQL gần với MySQL
- Có thêm một số kiểu dữ liệu riêng
- MVCC qua InnoDB / Aria

Nên chọn PostgreSQL, MySQL hay MariaDB?

SO SÁNH & BEST PRACTICES

POSTGRESQL

PHÙ HỢP KHI

- Dữ liệu và truy vấn phức tạp
- Cần toàn vẹn chặt, GIS hoặc phân tích

ĐIỂM MẠNH

- SQL chuẩn, JSONB, CTE, extension

LƯU Ý

- Cấu hình và tuning phức tạp hơn

MYSQL

PHÙ HỢP KHI

- Website CRUD phổ thông
- Hệ thống đọc nhiều, cấu trúc đơn giản

ĐIỂM MẠNH

- Dễ triển khai, hosting phổ biến

LƯU Ý

- Ít linh hoạt hơn với truy vấn nâng cao

MARIADB

PHÙ HỢP KHI

- Muốn hệ gần MySQL nhưng hướng cộng đồng
- Cần chuyển từ MySQL với ít thay đổi

ĐIỂM MẠNH

- Tương thích cao, thêm storage engine

LƯU Ý

- Không phải lúc nào cũng tương thích tuyệt đối

CHỌN NHANH: PostgreSQL = bài toán phức tạp | MySQL = web CRUD đơn giản | MariaDB = hệ MySQL hướng cộng đồng

Vì sao nhóm chọn PostgreSQL?

SO SÁNH & BEST PRACTICES

01 BÀI TOÁN CẦN GÌ?

- Dữ liệu quan hệ
- Ràng buộc chặt
- JOIN và giao dịch

02 POSTGRESQL ĐÁP ỨNG GÌ?

- SQL chuẩn + ACID
 - MVCC + Index
- JSONB + extension

03 ĐÁNH ĐỔI CHẤP NHẬN

- Cấu hình phức tạp hơn
- Cần hiểu role và pg_hba.conf
- Đổi lại để mở rộng dài hạn

KẾT LUẬN: PostgreSQL phù hợp nhất với demo CRUD, toàn vẹn dữ liệu và định hướng mở rộng lâu dài.

Lỗi thường gặp – quên GRANT

SO SÁNH & BEST PRACTICES



BEST PRACTICE

Cấp đúng quyền cho Role

ERROR: permission denied for table students

```
GRANT CONNECT ON DATABASE school TO app_user;  
GRANT USAGE ON SCHEMA public TO app_user;  
GRANT SELECT, INSERT, UPDATE, DELETE  
  ON ALL TABLES IN SCHEMA public TO app_user;  
  
ALTER DEFAULT PRIVILEGES IN SCHEMA public  
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_user;
```

✓ Cấp quyền hiện tại và quyền mặc định cho các bảng tạo sau này.

Kết luận & Key takeaways

TỔNG KẾT

- 01**
RDBMS mã nguồn mở
- Sát chuẩn SQL
 - ACID
 - Toàn vẹn dữ liệu

- 02**
Khái niệm cốt lõi
- Role
 - Database
 - Table
 - psql
 - Index
 - Backup

- 03**
Vòng đời demo
- Role + DB → CRUD → Index →
Backup → Restore

- 04**
Chọn hệ quản trị theo bài toán
- PostgreSQL: dữ liệu phức tạp
 - MySQL/MariaDB: web phổ thông

- 05**
Bài học vận hành
- GRANT đúng
 - pg_hba.conf đúng
 - Đúng format restore
 - Luôn có backup

References – official sources

TỔNG KẾT

CORE & SECURITY

- [1] PostgreSQL Global Development Group. PostgreSQL 18.4 Documentation.
<https://www.postgresql.org/docs/current/>
- [2] PostgreSQL Global Development Group. CREATE ROLE.
<https://www.postgresql.org/docs/current/sql-createrole.html>
- [3] PostgreSQL Global Development Group. The pg_hba.conf File.
<https://www.postgresql.org/docs/current/auth-pg-hba-conf.html>
- [4] PostgreSQL Global Development Group. Server Configuration.
<https://www.postgresql.org/docs/current/runtime-config.html>

OPERATIONS & PERFORMANCE

- [5] PostgreSQL Global Development Group. pg_dump.
<https://www.postgresql.org/docs/current/app-pgdump.html>
- [6] PostgreSQL Global Development Group. pg_restore.
<https://www.postgresql.org/docs/current/app-pgrestore.html>
- [7] PostgreSQL Global Development Group. CREATE INDEX.
<https://www.postgresql.org/docs/current/sql-createindex.html>
- [8] PostgreSQL Global Development Group. EXPLAIN.
<https://www.postgresql.org/docs/current/sql-explain.html>
- [9] PostgreSQL Global Development Group. MVCC: Introduction.
<https://www.postgresql.org/docs/current/mvcc-intro.html>

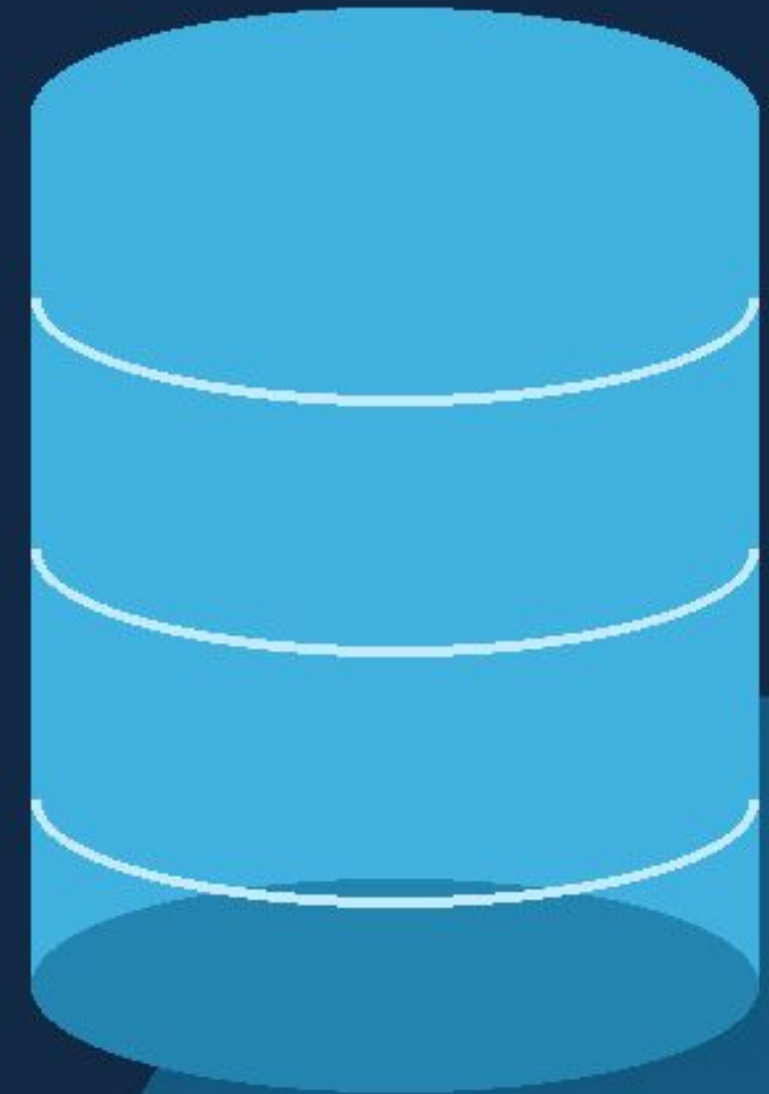
COMPARISON & TUTORIAL

- [10] Oracle. MySQL Documentation.
<https://dev.mysql.com/doc/>
- [11] MariaDB. MariaDB Server Documentation.
<https://mariadb.com/docs/server/>
- [12] PostgreSQL Global Development Group. PostgreSQL Tutorial.
<https://www.postgresql.org/docs/current/tutorial.html>

**Cảm ơn thầy cô và các bạn đã lắng
nghe!**



Q&A



POSTGRESQL